

MEMORIA PRÁCTICA 5

Antonio Morono
Pedro Ismael Haddou

Grupo 2272

Diseño de la práctica

Apartado 1: Monitorización de estados

Para responder al primer apartado, hemos creado la clase *ObjectStateTracker* que se parametriza con un tipo *T* (la clase de los objetos) y un tipo *E* (la clase para los estados). Para que los estados de tipo *E* definan un orden natural, en la definición de *ObjectStateTracker* nos aseguramos de que *E* extienda el tipo *Comparable<E>*. Definimos asimismo los atributos de la clase que nos permitirán implementar los métodos del enunciado:

- **mapa**: *Map<E, Predicate<T>>*. Se trata de un mapa que asocia a cada estado una expresión lambda tal que si se cumple, entonces el objeto está en este estado.
- **statesObjects**: *Map<E, List<T>>*. Se trata de un mapa que asocia a cada estado una lista de objetos que se encuentran en dicho estado.

A partir del primer atributo podemos implementar los métodos **withState** y **elseState**, que guardan el estado y la expresión lambda en **mapa**. Además, estos dos métodos lanzan la excepción **IllegalStateException** (que recibe un mensaje de excepción) cuando el estado de entrada no corresponde a ningún estado de la lista.

Con ayuda de los atributos, implementamos además el método **updateStates**, que dado el conjunto de objetos y de estados, evalúa la expresión lambda del estado a dicho objeto, y se comprueba si el objeto se encuentra en dicho estado. Pueden darse dos casos:

- Si el objeto estaba en dicho estado, y no acababa de ser añadido o la expresión lambda evalúa a **false**, el objeto deja de estar en ese estado.
- Si la expresión lambda evalúa a **true** y el objeto no está en el estado, entonces este objeto pasa a estar en dicho estado.

Por último, implementamos el método **addObjects** que aparece en *TesterStateChanges* y añade los objetos pasados como argumentos a un mapa **trajectorie**, que explicaremos más adelante.

Apartado 2: Igualdad de objetos

Con el fin de evitar almacenar objetos que se consideran iguales en la clase *ObjectStateTracker*, sobrescribimos los métodos **equals** y **hashCode** en la clase *Registration* de tal forma que las inscripciones con el mismo nombre sean iguales.

Apartado 3: Trayectoria de estados

Para llevar a cabo este tercer apartado e implementar el método **trajectory** que devuelve la trayectoria de un objeto, añadimos un nuevo atributo **trajectories** a la clase *ObjectStateTracker* que corresponde a un mapa que asocia un objeto a una lista de String. Dicha lista contiene las trayectorias en forma de String, donde se indica el origen y destino, y la fecha y hora del cambio. Lo que haremos con este atributo será añadirle una trayectoria al objeto correspondiente cuando cambie de estado. Para ello modificamos los anteriores

métodos de tal forma que **addObjects** añada el objeto a **trajectories** junto con una nueva lista de String cuyo primer valor será una trayectoria inicial que empieza con “in:”; también modificamos el método **updateStates** de tal forma que cuando el objeto pasa de un estado a otro, se añade a su lista de String del mapa **trajectories** una nueva trayectoria con “from: ... to...”.

Por último, implementamos el método **trajectory** que devuelve la trayectoria del objeto argumento, es decir, la lista de trayectorias que tiene asociada en el mapa.

Apartado 4: Procesos

Queremos construir una clase *Process* que guarde cierta información sobre cada posible estado a partir de las trayectorias de los objetos. Para ello, la clase tendrá tres atributos:

- **numInicial**: *Map<E, Integer>*. Se trata de un mapa que a cada estado se asocia el número de veces que ha sido un estado inicial en una trayectoria.
- **numFinal**: *Map<E, Integer>*. Semejante al anterior atributo salvo que guarda el número de veces que ha sido un estado final en una trayectoria.
- **numOtroEstado**: *Map<E, Map<E, Integer>>*. Corresponde a un mapa que asocia un estado con otro estado y el número de veces que ha habido una transición entre ellos.

Con estos atributos, podemos implementar el método **add** de la clase *Process* que, con la ayuda del método privado **stringToState**, guarda la información de la trayectoria pasada como argumento en los atributos.

Por otra parte, para facilitar la obtención de las trayectorias a partir de un objeto *ObjectStateTracker*, hacemos que esta clase implemente la interfaz *Iterable<T>* y sobreescribimos el método **iterator** tal que devuelva el iterador del mapa **trajectories**.

Apartado 5: Mejorando la usabilidad mediante el patrón Observer

Para implementar el patrón Observer en la práctica, basta con crear una nueva interfaz *IObserver* que tenga el método **updateStates**, y haremos que la clase *ObjectStateTracker* implemente dicha interfaz. Por último, la clase *Registration* tendrá un atributo de clase de tipo *IObserver<Registration>*, que será el observador. El método **withTracker** asociará el monitorizador que pasamos como argumento a este atributo. Añadiendo el método **updateStates** en todos los métodos de *Registration* que impliquen una modificación del objeto, se notificará a todos los objetos registrados, permitiendo así no tener que llamar de manera externa a **updateStates**.

Diagrama de clases

